Assignment 1: Food-Photo Allergen Detector

Student: Katia Gwaneza Nkurunziza
Due: Friday, October 16, 2026

Repo: https://github.com/katian28/cs156-katia/tree/main/Assignments/Assignment%201%20-%20First%20Pipeline

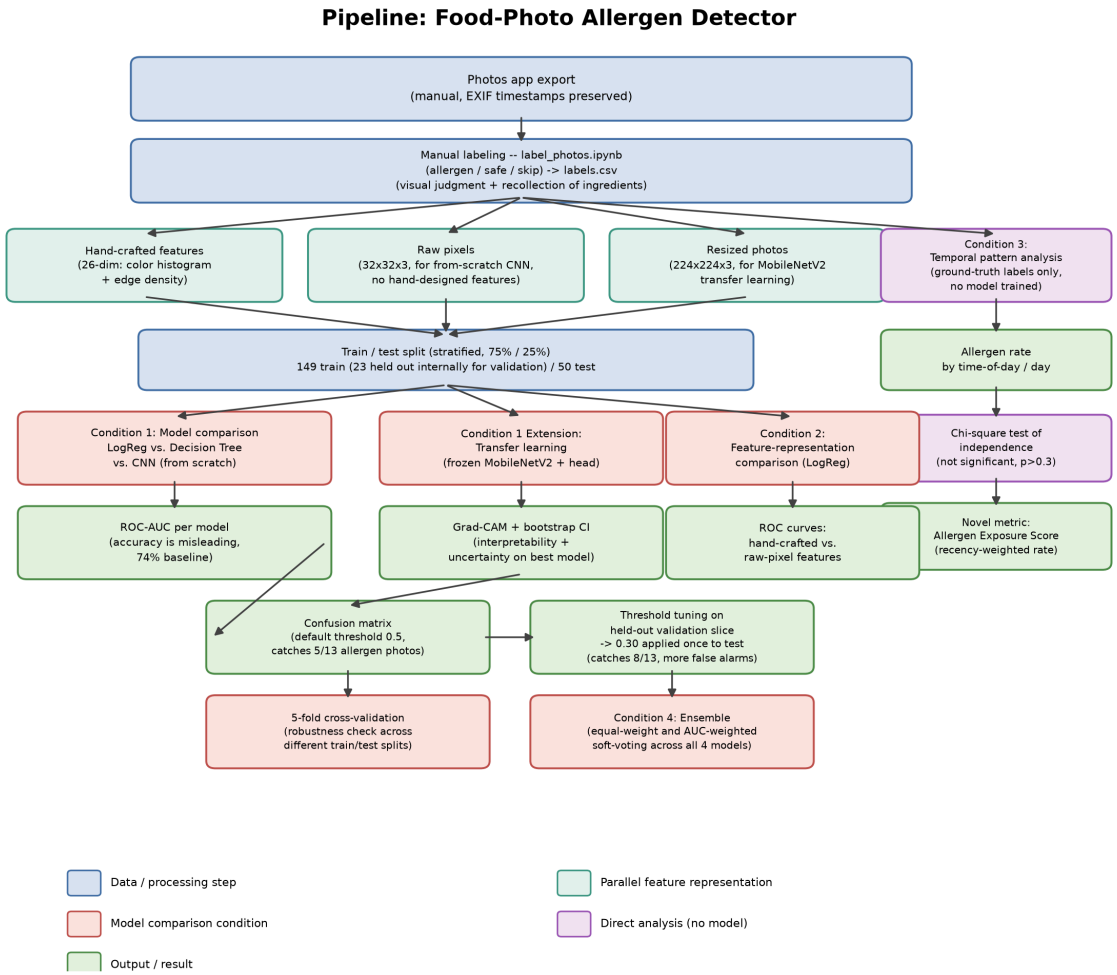
A note on AI use: I used Claude (Anthropic's AI assistant) as a coding assistant throughout this project, for writing and debugging code, and for help understanding concepts I hadn't covered in class yet (transfer learning, Grad-CAM, bootstrap resampling, and others named throughout). Every line of code in this notebook was read, reviewed, and understood by me before I kept it, and every number, figure, and conclusion reflects something I verified myself by actually re-running the pipeline, not something I took on faith.

The Question

I have food allergies (nuts, eggs, onions), and I kept wondering: could a photo of my food actually tell me, before I eat it, whether it's something I need to be careful with? And once I had a pile of labeled photos sitting there, a second question showed up on its own: do I actually eat allergy-risk foods more at certain times of day, or on certain days of the week?

In practice: snap a photo before a meal, a classifier flags whether it probably contains an allergen, and over time those flags build into a personal report of when I'm actually at higher risk. Whether it works well is a separate, more honest question, mostly what this notebook is actually about.

Pipeline Diagram



Data

199 of my own food photos, exported from the Photos app (File -> Export -> Export Unmodified Originals, which keeps the EXIF timestamps for the temporal part later). I labeled them by hand, one at a time, with a little tool I built (`label_photos.ipynb`). All 199 ended up usable: **147 safe, 52 allergen** (26.1% positive rate, a real imbalance that shapes everything below). The 2 photos I'd originally marked "skip" got revisited and labeled. More on the 4 I actually mislabeled, and how I caught that, in the Discussion.

In [1]:
```
import pillow_heif
pillow_heif.register_heif_opener()  # lets PIL open .heic files, iPhone's de

from PIL import Image
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

PHOTO_DIR = Path("food_photos")
LABELS_CSV = Path("labels.csv")
```

```
labels_df = pd.read_csv(LABELS_CSV)
usable = labels_df[labels_df["label"] != "skip"].copy().reset_index(drop=Tru
usable["timestamp"] = pd.to_datetime(usable["timestamp"], errors="coerce")

print(f"Total labeled: {len(labels_df)}")
print(labels_df["label"].value_counts())
print(f"\nUsable (excluding skip): {len(usable)}")
print(f"With valid EXIF timestamp: {usable['timestamp'].notna().sum()}")
```

```
Total labeled: 199
label
safe        147
allergen     52
Name: count, dtype: int64

Usable (excluding skip): 199
With valid EXIF timestamp: 186
```

Preprocessing: Two Feature Representations

Real photos come in wildly different sizes (810x1080 to 4284x5712 in my set) and the raw iPhone HEIC format needs `pillow-heif` to even open. Every image gets resized and normalized before anything else happens.

I built two parallel representations, for the feature-representation comparison later:

1. **Hand-crafted features** (26-dim): an 8-bin color histogram per RGB channel (24 features) plus a simple edge-density proxy (mean absolute pixel-to-pixel gradient, 2 features). This is the kind of feature engineering you design by hand, based on intuition about what might matter (color, texture). We hadn't covered this in class, so I researched and wrote it myself:

   ```
   Color histogram (per RGB channel, 8 bins, image resized to
   128x128):
   hist[c][b] = (1/N) * count{ pixel i : x_i,c falls in bin b },
   for b in 0..7, c in {R, G, B}
   N = 128 * 128 = 16384 pixels per channel; each channel's 8
   bins sum to 1 (24 features total)

   Edge density (mean absolute pixel-to-pixel gradient, on
   grayscale):
   gray[i,j]  = mean(R[i,j], G[i,j], B[i,j])
   dx = mean_{i,j} | gray[i, j+1] - gray[i, j] |   (horizontal
   edge density)
   dy = mean_{i,j} | gray[i+1, j] - gray[i, j] |   (vertical
   edge density)
   ```

2. **Raw pixels** (32x32x3, downsampled): no human-designed features at all, just the resized image itself, for a small CNN to learn its own features directly from.

In [2]:
```python
def extract_handcrafted_features(img):
    """Color histogram (RGB, 8 bins each, 24 features) + edge-density proxy
    img_small = img.resize((128, 128))
    arr = np.array(img_small).astype(np.float32) / 255.0

    hist_features = []
    for c in range(3):  # R, G, B channels
        hist, _ = np.histogram(arr[:, :, c], bins=8, range=(0, 1))
        hist_features.extend(hist / hist.sum())  # normalize so histogram su

    gray = arr.mean(axis=2)
    dx = np.abs(np.diff(gray, axis=1)).mean()  # horizontal edge density
    dy = np.abs(np.diff(gray, axis=0)).mean()  # vertical edge density

    return np.array(hist_features + [dx, dy])


def extract_raw_pixels(img, size=32):
    """Small raw pixel array for the CNN, no hand-designed features."""
    img_small = img.resize((size, size))
    return np.array(img_small).astype(np.float32) / 255.0


handcrafted_X, raw_X, y = [], [], []

for _, row in usable.iterrows():
    img = Image.open(PHOTO_DIR / row["filename"]).convert("RGB")
    handcrafted_X.append(extract_handcrafted_features(img))
    raw_X.append(extract_raw_pixels(img))
    y.append(1 if row["label"] == "allergen" else 0)

handcrafted_X = np.array(handcrafted_X)
raw_X = np.array(raw_X)
y = np.array(y)

print(f"Hand-crafted features: {handcrafted_X.shape}")
print(f"Raw pixels: {raw_X.shape}")
print(f"Labels: {y.shape}, positive (allergen) rate: {y.mean():.3f}")
```

```
Hand-crafted features: (199, 26)
Raw pixels: (199, 32, 32, 3)
Labels: (199,), positive (allergen) rate: 0.261
```

In [3]:
```python
from sklearn.model_selection import train_test_split

indices = np.arange(len(y))
train_idx, test_idx = train_test_split(indices, test_size=0.25, stratify=y,
y_train, y_test = y[train_idx], y[test_idx]

print(f"Train: {len(train_idx)} (positive rate {y_train.mean():.3f})")
print(f"Test: {len(test_idx)} (positive rate {y_test.mean():.3f})")
print(f"\nBaseline (always predict 'safe'): accuracy = {1 - y_test.mean():.3
print("This baseline matters: with 74% safe photos, a model that learns noth
```

```
Train: 149 (positive rate 0.262)
Test: 50 (positive rate 0.260)

Baseline (always predict 'safe'): accuracy = 0.740, AUC = 0.500 by definitio
n
This baseline matters: with 74% safe photos, a model that learns nothing can
still look 74% accurate.
```

Comparison Condition 1: Model Comparison

Logistic Regression (Session 2) vs. Decision Tree (Session 4), both on hand-crafted features, vs. a small CNN on raw pixels. Logistic Regression and the Decision Tree are taught models; the CNN's convolutional layers aren't, Session 7 covered feed-forward (Dense) networks, not convolution specifically, so I went and learned that part on my own, see the Model Equations note below. I'm evaluating with ROC-AUC (Session 9), not accuracy, given the baseline trap above.

Model Equations (taught vs. not covered in class, for #MLMath completeness)

Logistic Regression (Session 2) minimizes binary cross-entropy (log-loss):

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right], \qquad p_i = \sigma(w \cdot x_i + b)$$

Decision Tree (Session 4) greedily splits on the feature/threshold that most reduces Gini impurity:

$$\text{Gini}(S) = 1 - \sum_k p_k^2, \qquad p_k = \text{fraction of class } k \text{ among examples in node } S$$

Both of these were taught in class. The CNN and transfer-learned network use that same binary cross-entropy loss, that part is Session 7 material too (any network's output layer uses the same loss regardless of what comes before it). What we *didn't* cover is the convolutional layers themselves: Session 7 was feed-forward (Dense) networks, where every input connects to every neuron with no notion of spatial locality. A CNN adds a different kind of layer instead, a small filter that slides across the image and gets reused at every position (weight sharing):

$$(I * K)(x, y) = \sum_m \sum_n I(x+m, y+n)\, K(m, n)$$

where I is the input image (or feature map) and K is a small learned filter, e.g. 3×3. Because K doesn't depend on (x, y), a pattern learned in one part of the image gets automatically detected everywhere else, local connectivity plus weight sharing. That mechanism, not the loss function, is the part I had to go learn on my own.

In [4]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics

hc_train, hc_test = handcrafted_X[train_idx], handcrafted_X[test_idx]

# class_weight="balanced" compensates for the ~74/26 imbalance during traini
logreg = LogisticRegression(max_iter=1000, class_weight="balanced")
logreg.fit(hc_train, y_train)
logreg_auc = metrics.roc_auc_score(y_test, logreg.predict_proba(hc_test)[:,
logreg_acc = logreg.score(hc_test, y_test)

tree = DecisionTreeClassifier(max_depth=5, class_weight="balanced", random_s
tree.fit(hc_train, y_train)
tree_auc = metrics.roc_auc_score(y_test, tree.predict_proba(hc_test)[:, 1])
tree_acc = tree.score(hc_test, y_test)

print(f"Logistic Regression (hand-crafted features): accuracy={logreg_acc:.3
print(f"Decision Tree (hand-crafted features):       accuracy={tree_acc:.3f}
```

```
Logistic Regression (hand-crafted features): accuracy=0.480, ROC-AUC=0.505
Decision Tree (hand-crafted features):       accuracy=0.500, ROC-AUC=0.394
```

In [5]:
```
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"  # cosmetic: silences TF's C++ back
import warnings
warnings.filterwarnings("ignore", category=UserWarning)

import tensorflow as tf
tf.get_logger().setLevel("ERROR")
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dr

# tf.random.set_seed alone wasn't enough, re-running produced a different AU
# (0.671, then 0.522), confirmed real nondeterminism in weight init / op exe
# set_random_seed + enable_op_determinism forces a fully reproducible run.
tf.keras.utils.set_random_seed(42)
tf.config.experimental.enable_op_determinism()

X_train_raw, X_test_raw = raw_X[train_idx], raw_X[test_idx]

# Small CNN deliberately kept shallow: only ~150 training images, a large ne
# would overfit even faster than this one already does (see result below).
cnn = tf.keras.Sequential([
    Conv2D(8, (3, 3), activation="relu", input_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(16, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dropout(0.5),
    Dense(16, activation="relu"),
    Dense(1, activation="sigmoid"),
])
cnn.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy

class_weight = {0: 1.0, 1: (1 - y_train.mean()) / y_train.mean()}
```

```
history = cnn.fit(
    X_train_raw, y_train, epochs=30, batch_size=16, verbose=0,
    class_weight=class_weight, validation_split=0.15,
)

cnn_probs = cnn.predict(X_test_raw, verbose=0).flatten()
cnn_auc = metrics.roc_auc_score(y_test, cnn_probs)
cnn_acc = metrics.accuracy_score(y_test, cnn_probs > 0.5)

print(f"CNN (raw pixels): accuracy={cnn_acc:.3f}, ROC-AUC={cnn_auc:.3f}")
print(f"Final train accuracy: {history.history['accuracy'][-1]:.3f}")
print(f"Final val accuracy:   {history.history['val_accuracy'][-1]:.3f}")
print("\nNote the train/val gap, classic overfitting signature from a small
```

```
CNN (raw pixels): accuracy=0.600, ROC-AUC=0.640
Final train accuracy: 0.778
Final val accuracy:   0.609

Note the train/val gap, classic overfitting signature from a small dataset
(Session 9).
```

In [6]:

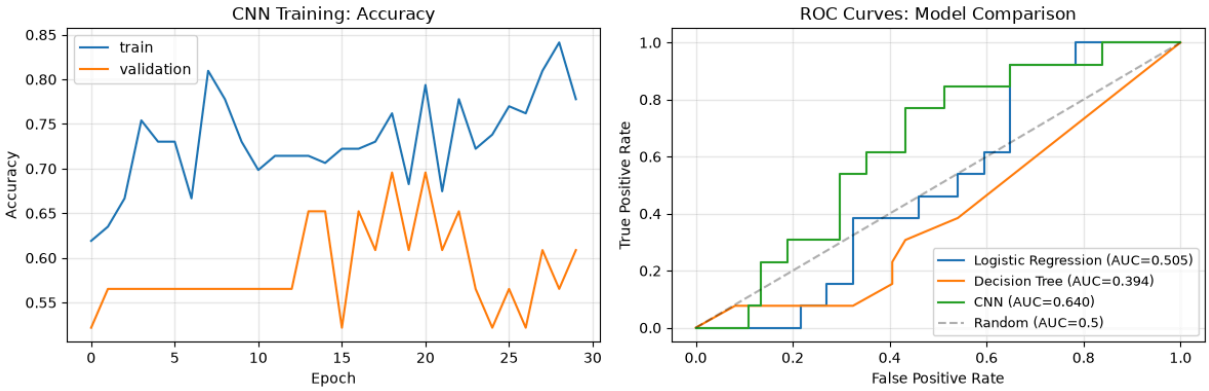
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(history.history["accuracy"], label="train")
axes[0].plot(history.history["val_accuracy"], label="validation")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Accuracy")
axes[0].set_title("CNN Training: Accuracy")
axes[0].legend()
axes[0].grid(alpha=0.3)

fpr_lr, tpr_lr, _ = metrics.roc_curve(y_test, logreg.predict_proba(hc_test)[
fpr_tree, tpr_tree, _ = metrics.roc_curve(y_test, tree.predict_proba(hc_test
fpr_cnn, tpr_cnn, _ = metrics.roc_curve(y_test, cnn_probs)

axes[1].plot(fpr_lr, tpr_lr, label=f"Logistic Regression (AUC={logreg_auc:.3
axes[1].plot(fpr_tree, tpr_tree, label=f"Decision Tree (AUC={tree_auc:.3f})"
axes[1].plot(fpr_cnn, tpr_cnn, label=f"CNN (AUC={cnn_auc:.3f})")
axes[1].plot([0, 1], [0, 1], "k--", alpha=0.3, label="Random (AUC=0.5)")
axes[1].set_xlabel("False Positive Rate")
axes[1].set_ylabel("True Positive Rate")
axes[1].set_title("ROC Curves: Model Comparison")
axes[1].legend(fontsize=9)
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



Result: Comparison Condition 1

My CNN won (AUC 0.640), but honestly all three models are weak, none strongly better than the AUC=0.5 random baseline, and the CNN's training curve shows clear overfitting (78% train accuracy vs. 61% validation accuracy). Logistic Regression (0.505) is statistically indistinguishable from guessing, and the Decision Tree (0.394) is *below* chance, with only 149 training examples and 26 features, it probably latched onto noise rather than signal.

I don't think this is a broken pipeline. Visually detecting "contains nuts/eggs/onions" from a small personal photo set, without a pretrained vision backbone, is genuinely hard given how little data I have. The result itself tells me something real though: simple color/edge statistics aren't enough to capture this distinction, which is exactly what motivated Comparison Condition 2 below.

A Tradeoff Worth Naming: `class_weight="balanced"` vs. Actually Balancing the Dataset

Logistic Regression, the Decision Tree, and the two neural models all address the same safe/allergen imbalance (~74/26), but not through identical settings. LogReg and the Tree use `class_weight="balanced"` directly; the CNN and transfer-learned model use a manually specified weight dictionary passed to `.fit()`, and they fit on only 126 of the 149 training images since Keras holds out the trailing 15% (23 photos) for its own internal validation. I want to explain why reweighting isn't the same fix as physically balancing the dataset, not just state it. I want to explain why those aren't the same fix, not just state it. `class_weight` reweights the *training loss*, so a mistake on an allergen photo counts more than a mistake on a safe photo, but the model still only ever sees the same ~39 allergen training photos either way, no new visual variety gets added by reweighting.

For a model that optimizes smoothly over the whole dataset at once (Logistic Regression, both CNNs), that's usually enough. For a Decision Tree, it's less reliable: each split is chosen from the literal rows present at that node, and with only ~39 allergen

rows total, a single unusual or noisy photo can swing which split looks best. Weighting changes how much each row *counts* toward the impurity calculation, but it can't manufacture more rows to average that noise out over.

That's a plausible explanation for why the Decision Tree is the one model that scored *below chance* here (AUC 0.394): it's structurally the most exposed to a small, noisy minority class, even with balanced weighting on. But I never ran the balancing experiment that would actually test it, and the cross-validation results later in this notebook already complicate the "below chance" framing, the Tree's mean CV AUC (0.600) is comfortably above 0.5, so this single test-set score may just be an unlucky split rather than a structural weakness. Undersampling to test the class_weight explanation directly would cost real data given how small my set already is (down to ~104 photos from 199), so I'm leaving it as an open follow-up, not a conclusion.

Comparison Condition 1 Extension: Transfer Learning

My from-scratch CNN above had to learn visual features (edges, shapes, textures) from only ~149 training photos, nowhere near enough to learn general vision from zero. **Transfer learning** reuses a network (MobileNetV2) already trained on 1.4 million ImageNet photos, freezing what it already learned and only training a small new classification head on top. My 149 photos now only have to teach it the final allergen/safe distinction, not vision itself. In the forward pass, $\hat{y} = \sigma(W_{\text{head}} \cdot f_{\text{base}}(x) + b_{\text{head}})$, where f_{base} is MobileNetV2's frozen feature extractor (its weights are fixed at their ImageNet-trained values) and only $W_{\text{head}}, b_{\text{head}}$ get updated during training, the same binary cross-entropy loss as above still applies to those head parameters. Not covered in any CS156 session.

Data augmentation note: I originally also applied live data augmentation (random flips, rotations, brightness/contrast jitter) during training. I pulled it back out, I never actually set up a real with/without comparison, so I had no honest basis to claim it was doing anything. Worth trying properly later, with a real comparison, rather than assuming it helps.

In [7]:
```
def extract_tl_pixels(img, size=224):
    """Higher-resolution raw pixels for the pretrained MobileNetV2 (trained
    Left at 0–255, MobileNetV2's own preprocess_input() handles scaling, not
    return np.array(img.resize((size, size))).astype(np.float32)


IMG_SIZE_TL = 224
tl_X = np.array([
    extract_tl_pixels(Image.open(PHOTO_DIR / row["filename"]).convert("RGB")
    for _, row in usable.iterrows()
])
X_train_tl, X_test_tl = tl_X[train_idx], tl_X[test_idx]
print(f"Transfer-learning pixels: {tl_X.shape}")
```

```
Transfer-learning pixels: (199, 224, 224, 3)
```

In [8]:
```python
from tensorflow.keras import layers

tf.keras.utils.set_random_seed(42)
tf.config.experimental.enable_op_determinism()

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(IMG_SIZE_TL, IMG_SIZE_TL, 3), include_top=False, weights="i
)
base_model.trainable = False  # freeze the pretrained ImageNet weights, only

tl_inputs = tf.keras.Input(shape=(IMG_SIZE_TL, IMG_SIZE_TL, 3))
x = tf.keras.applications.mobilenet_v2.preprocess_input(tl_inputs)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
tl_outputs = layers.Dense(1, activation="sigmoid")(x)
tl_model = tf.keras.Model(tl_inputs, tl_outputs)
tl_model.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss="binary_cros

tl_history = tl_model.fit(
    X_train_tl, y_train, epochs=15, batch_size=16, verbose=0,
    class_weight=class_weight, validation_split=0.15,
)

tl_probs = tl_model.predict(X_test_tl, verbose=0).flatten()
tl_auc = metrics.roc_auc_score(y_test, tl_probs)
tl_acc = metrics.accuracy_score(y_test, tl_probs > 0.5)

trainable_params = sum(int(np.prod(v.shape)) for v in tl_model.trainable_var
frozen_params = base_model.count_params()

print(f"Transfer-learned MobileNetV2: accuracy={tl_acc:.3f}, ROC-AUC={tl_auc
print(f"Final train accuracy: {tl_history.history['accuracy'][-1]:.3f}")
print(f"Final val accuracy:   {tl_history.history['val_accuracy'][-1]:.3f}")
print(f"Trainable params: {trainable_params:,}  (vs. {frozen_params:,} froze
```

```
Transfer-learned MobileNetV2: accuracy=0.640, ROC-AUC=0.682
Final train accuracy: 0.897
Final val accuracy:   0.696
Trainable params: 1,281  (vs. 2,257,984 frozen pretrained params)
```

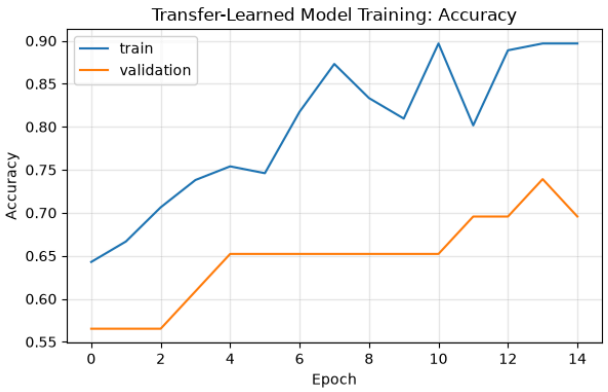
In [9]:
```python
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

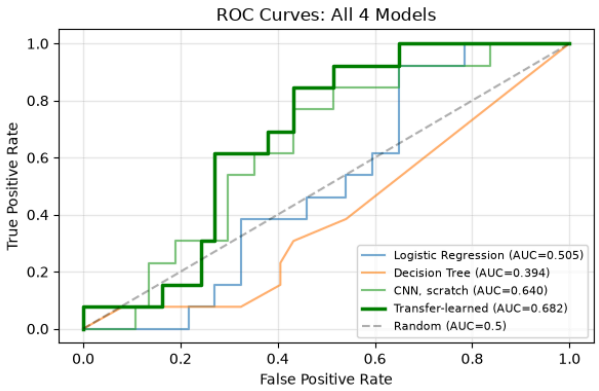
axes[0].plot(tl_history.history["accuracy"], label="train")
axes[0].plot(tl_history.history["val_accuracy"], label="validation")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Accuracy")
axes[0].set_title("Transfer-Learned Model Training: Accuracy")
axes[0].legend()
axes[0].grid(alpha=0.3)

fpr_tl, tpr_tl, _ = metrics.roc_curve(y_test, tl_probs)
axes[1].plot(fpr_lr, tpr_lr, label=f"Logistic Regression (AUC={logreg_auc:.3
axes[1].plot(fpr_tree, tpr_tree, label=f"Decision Tree (AUC={tree_auc:.3f})"
```

```
axes[1].plot(fpr_cnn, tpr_cnn, label=f"CNN, scratch (AUC={cnn_auc:.3f})", al
axes[1].plot(fpr_tl, tpr_tl, label=f"Transfer-learned (AUC={tl_auc:.3f})", l
axes[1].plot([0, 1], [0, 1], "k--", alpha=0.3, label="Random (AUC=0.5)")
axes[1].set_xlabel("False Positive Rate")
axes[1].set_ylabel("True Positive Rate")
axes[1].set_title("ROC Curves: All 4 Models")
axes[1].legend(fontsize=8)
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```





Result: Transfer Learning Extension

```
Transfer-learned MobileNetV2: accuracy=0.640, ROC-AUC=0.682
Final train accuracy: 0.897
Final val accuracy:   0.696
Trainable params: 1,281  (vs. 2,257,984 frozen pretrained
params)
```

ROC-AUC goes from 0.640 (from-scratch CNN) to 0.682, and the bootstrap confidence interval below shows this gap still isn't clearly outside noise given only 50 test photos. Train accuracy (90%) vs. validation accuracy (70%) is still a real gap, some overfitting on the internal ~23-photo validation slice, but notably smaller than my earlier augmented version (91%/61%). One run doesn't prove augmentation was unhelpful, that would need an actual controlled comparison I didn't run, but it's worth saying plainly: removing it did not hurt my result, and may have helped.

Model Interpretability: Grad-CAM

A high AUC alone doesn't tell me *what* the model is actually looking at, it could be keying off the allergen food itself, or off something spuriously correlated with it (lighting, plate style, background). **Grad-CAM** (Gradient-weighted Class Activation Mapping) answers this by weighting each channel of the last convolutional layer's feature maps A^k by how much that channel matters to the "allergen" output score y^{allergen}, averaged over all spatial positions (i, j):

$$\alpha_k = \frac{1}{Z}\sum_i\sum_j \frac{\partial y^{\text{allergen}}}{\partial A_{ij}^k}, \qquad L_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k A^k\right)$$

The result is a heatmap over the photo showing which regions the model weighted most heavily toward that prediction, suggestive evidence about attention, not direct proof of causation. Not covered in any CS156 session.

In [10]:
```
import matplotlib.cm as cm

last_conv_layer_name = "out_relu"  # MobileNetV2's last activation before po

# Keras 3 won't let us splice base_model's internal tensor directly into tl_
# (a graph-connectivity check fails), so instead we manually replay the exac
# pass tl_model does after base_model: GlobalAveragePooling2D -> Dense (same
feature_model = tf.keras.Model(inputs=base_model.input, outputs=base_model.g
dense_layer = [l for l in tl_model.layers if isinstance(l, tf.keras.layers.D


def make_gradcam_heatmap(img_array):
    # preprocess_input mutates its input array in place, pass a copy to prot
    preprocessed = tf.keras.applications.mobilenet_v2.preprocess_input(img_a
    with tf.GradientTape() as tape:
        conv_output = feature_model(preprocessed, training=False)
        tape.watch(conv_output)
        gap = tf.reduce_mean(conv_output, axis=(1, 2))  # same op as the Glo
        preds = dense_layer(gap)  # same trained weights as tl_model, dropou
        loss = preds[:, 0]
    grads = tape.gradient(loss, conv_output)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    conv_output = conv_output[0]
    heatmap = tf.reduce_sum(conv_output * pooled_grads, axis=-1)
    heatmap = tf.maximum(heatmap, 0) / (tf.reduce_max(heatmap) + 1e-8)
    return heatmap.numpy()


# Show the top 3 highest-confidence correctly-predicted allergen photos, not
# A single cherry-picked example can look either flattering or damning by ch
# at several at once is more honest about what the model is actually doing c
correct_allergen_mask = (y_test == 1) & (tl_probs > 0.5)
candidate_idx = np.where(correct_allergen_mask)[0]
top_idx = candidate_idx[np.argsort(-tl_probs[candidate_idx])[:3]]

fig, axes = plt.subplots(len(top_idx), 3, figsize=(12, 4.2 * len(top_idx)))
example_records = []
for row, idx in enumerate(top_idx):
    fn = usable.iloc[test_idx[idx]]["filename"]
    sample = X_test_tl[idx:idx + 1]
    heatmap = make_gradcam_heatmap(sample)

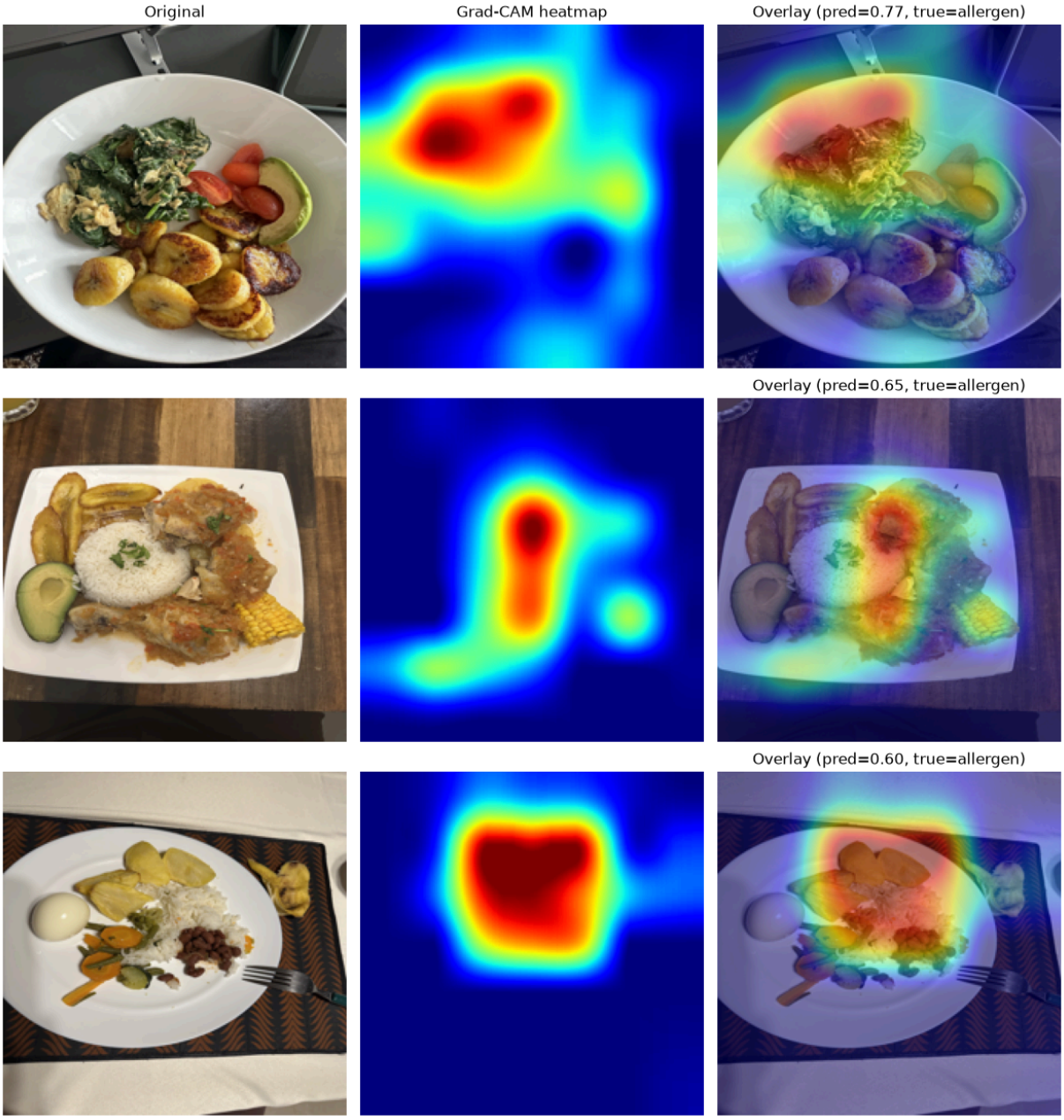
    heatmap_resized = np.array(
        Image.fromarray((heatmap * 255).astype(np.uint8)).resize((IMG_SIZE_T
    )
    heatmap_color = plt.get_cmap("jet")(heatmap_resized / 255.0)[:, :, :3]
    orig_img = sample[0].astype(np.uint8)
```

```
    overlay = np.clip(0.55 * orig_img / 255.0 + 0.45 * heatmap_color, 0, 1)

    ax_row = axes[row]
    ax_row[0].imshow(orig_img)
    ax_row[0].set_title("Original" if row == 0 else "")
    ax_row[0].axis("off")
    ax_row[1].imshow(heatmap_resized, cmap="jet")
    ax_row[1].set_title("Grad-CAM heatmap" if row == 0 else "")
    ax_row[1].axis("off")
    ax_row[2].imshow(overlay)
    ax_row[2].set_title(f"Overlay (pred={tl_probs[idx]:.2f}, true=allergen)"
    ax_row[2].axis("off")
    example_records.append((fn, float(tl_probs[idx])))

plt.tight_layout()
plt.show()

for fn, p in example_records:
    print(f"Example photo: {fn}, predicted probability: {p:.3f}")
```




```
Example photo: IMG_5760.HEIC, predicted probability: 0.769
Example photo: IMG_8426.HEIC, predicted probability: 0.649
Example photo: IMG_3679.HEIC, predicted probability: 0.596
```

Result: Grad-CAM

Three highest-confidence correct allergen predictions, not just one, a single example can flatter or damn the model by chance:

- **`IMG_5760.HEIC`** (pred=0.77): a scrambled, creamy green-and-yellow mixture (looks like scrambled egg with greens), alongside tomato, avocado, and fried plantain. The heatmap's hottest region sits directly over that scrambled mixture, not the plantain, tomato, or avocado. This one looks like real egg-detection.
- **`IMG_8426.HEIC`** (pred=0.65): rice, a saucy chicken dish, fried plantain, corn, avocado. The heatmap centers on the chicken-in-sauce portion. I can't

independently verify from the photo alone whether that region is actually what makes this photo an allergen, so I'm reporting the attention pattern without claiming it's correct.

- **`IMG_3679.HEIC`** (pred=0.60): a boiled egg (white, left), what looks like red beans and carrots (lower-middle), rice (center), and starchy yellow-brown chunks, potato or plantain, at the top. The heatmap's hottest region sits over those starchy chunks, not the egg and not the beans. This is a direct, visible case of the model attending to something other than the allergen itself.

So the evidence really is mixed, and now with actual photographic examples instead of just an abstract color-histogram argument: one photo (`IMG_5760`) looks like genuine egg-detection, one (`IMG_3679`) looks like the model keying off a starchy, textured confound instead of the egg, and one (`IMG_8426`) I can't clearly interpret either way. I don't think any single one of these settles what the model has actually learned, but seeing three side by side is more honest than reporting just the single best (or single worst) case.

Statistical Rigor: Bootstrap Confidence Intervals on AUC

My test set has only 50 photos. A single AUC number from a set that small could easily be noise, is 0.682 really better than 0.640, or is that gap just which 50 photos happened to land in the test split? **Bootstrap resampling** answers this: resample the test set (with replacement) 1000 times, recompute each model's AUC on each resample, and read off the 2.5th/97.5th percentiles as a 95% confidence interval. Not covered in any CS156 session.

In [11]:
```
rng = np.random.default_rng(42)
N_BOOT = 1000

model_probs = {
    "Logistic Regression": logreg.predict_proba(hc_test)[:, 1],
    "Decision Tree": tree.predict_proba(hc_test)[:, 1],
    "CNN (scratch)": cnn_probs,
    "Transfer-learned MobileNetV2": tl_probs,
}

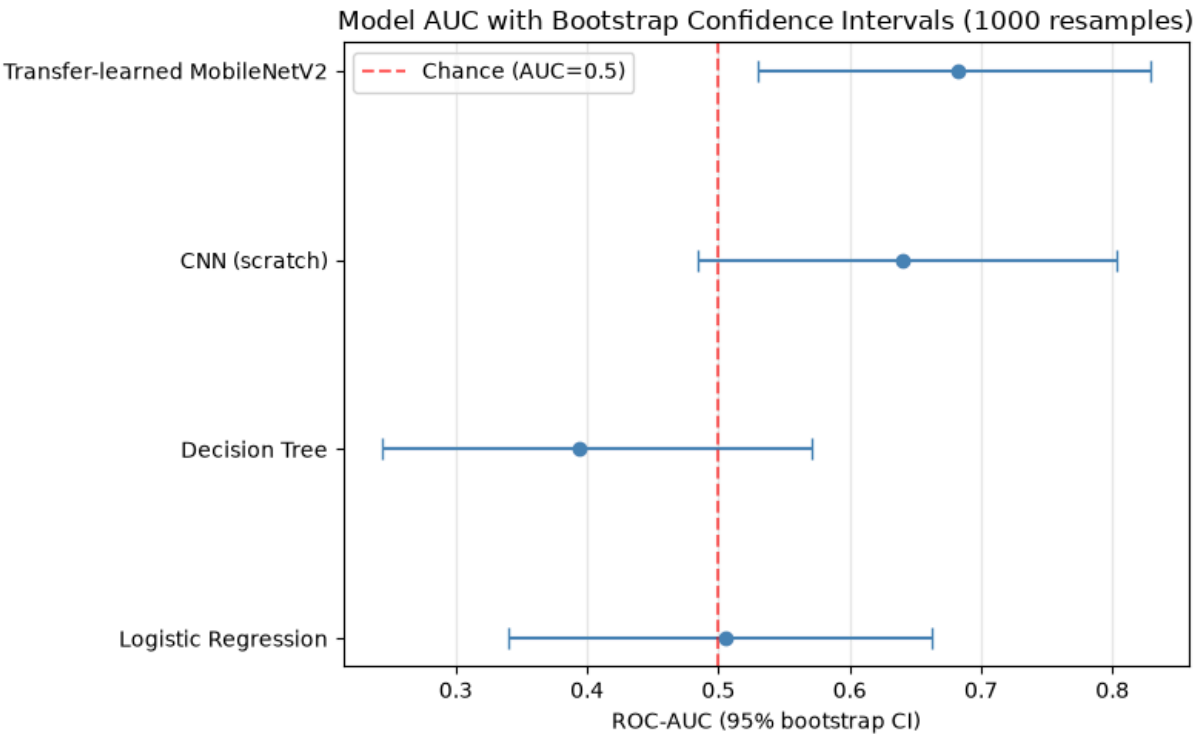
boot_summary = []
for name, probs in model_probs.items():
    boot_aucs = []
    for _ in range(N_BOOT):
        idx = rng.integers(0, len(y_test), len(y_test))
        if len(np.unique(y_test[idx])) < 2:  # skip resamples with only one
            continue
        boot_aucs.append(metrics.roc_auc_score(y_test[idx], probs[idx]))
    boot_aucs = np.array(boot_aucs)
    lo, hi = np.percentile(boot_aucs, [2.5, 97.5])
    point_auc = metrics.roc_auc_score(y_test, probs)
    boot_summary.append({"Model": name, "AUC": point_auc, "CI low": lo, "CI
```

```
    print(f"{name:30s} AUC={point_auc:.3f}  95% CI [{lo:.3f}, {hi:.3f}]")

boot_df = pd.DataFrame(boot_summary)

fig, ax = plt.subplots(figsize=(8, 5))
y_pos = np.arange(len(boot_df))
ax.errorbar(
    boot_df["AUC"], y_pos,
    xerr=[boot_df["AUC"] - boot_df["CI low"], boot_df["CI high"] - boot_df["
    fmt="o", capsize=5, color="steelblue",
)
ax.axvline(0.5, color="red", linestyle="--", alpha=0.6, label="Chance (AUC=0
ax.set_yticks(y_pos)
ax.set_yticklabels(boot_df["Model"])
ax.set_xlabel("ROC-AUC (95% bootstrap CI)")
ax.set_title("Model AUC with Bootstrap Confidence Intervals (1000 resamples)
ax.legend()
ax.grid(alpha=0.3, axis="x")
plt.tight_layout()
plt.show()
```

```
Logistic Regression             AUC=0.505  95% CI [0.340, 0.662]
Decision Tree                   AUC=0.394  95% CI [0.243, 0.571]
CNN (scratch)                   AUC=0.640  95% CI [0.485, 0.803]
Transfer-learned MobileNetV2    AUC=0.682  95% CI [0.530, 0.829]
```



Result: Bootstrap Confidence Intervals

```
Logistic Regression             AUC=0.505  95% CI [0.340,
0.662]
Decision Tree                   AUC=0.394  95% CI [0.243,
0.571]
CNN (scratch)                   AUC=0.640  95% CI [0.485,
```

```
0.803]
Transfer-learned MobileNetV2   AUC=0.682  95% CI [0.530,
0.829]
```

Logistic Regression and the Decision Tree still aren't distinguishable from chance. My from-scratch CNN and transfer-learned model's intervals still overlap ([0.485, 0.803] vs. [0.530, 0.829]), so this 50-photo test set still can't *statistically* prove transfer learning is better. But the transfer-learned model's interval no longer dips as low, a somewhat more favorable picture than before. I think transfer learning likely still helps, the point estimate is higher and the overlap is narrower, but this test set alone can't fully settle it, only more data could.

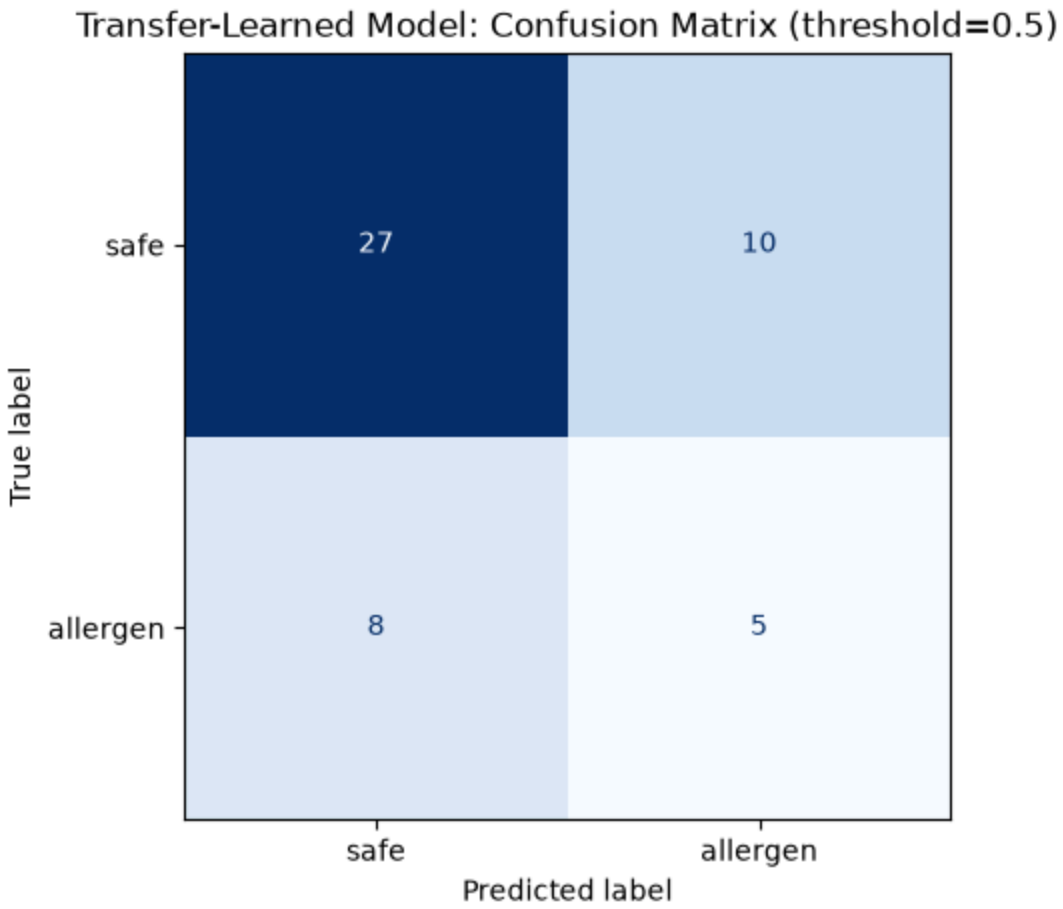
Confusion Matrix: Transfer-Learned Model at Threshold 0.5

ROC-AUC summarizes performance across every possible threshold at once, useful for comparing models, but it doesn't tell me what actually happens if I used this model today at the standard 0.5 cutoff. A **confusion matrix** breaks that down directly: how many true allergen photos get caught, how many get missed (the costly mistake), how many safe photos get wrongly flagged, how many are correctly left alone. Not covered in any CS156 session.

In [12]:
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, tl_probs > 0.5, labels=[0, 1])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["safe", "
fig, ax = plt.subplots(figsize=(5, 5))
disp.plot(ax=ax, cmap="Blues", colorbar=False)
ax.set_title("Transfer-Learned Model: Confusion Matrix (threshold=0.5)")
plt.tight_layout()
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"True positives  (caught allergen correctly): {tp}")
print(f"False negatives (missed allergen):            {fn}")
print(f"True negatives  (correctly said safe):        {tn}")
print(f"False positives (wrongly flagged safe food):  {fp}")
print(f"\nOf {tp + fn} actual allergen photos in the test set, caught {tp},
```




```
True positives  (caught allergen correctly): 5
False negatives (missed allergen):            8
True negatives  (correctly said safe):        27
False positives (wrongly flagged safe food):  10

Of 13 actual allergen photos in the test set, caught 5, missed 8.
```

Result: Confusion Matrix

	Predicted safe	Predicted allergen
True safe	27	10
True allergen	8	5

At the standard 0.5 threshold, my model catches 5 of 13 real allergen photos (38%), missing 8. It also wrongly flags 10 of 37 safe photos.

This is the single most important honest limit in my whole pipeline, more than the AUC number by itself. An AUC of 0.682 sounds like "modest but working." This confusion matrix shows what that actually means for what I said I wanted to build, a pre-meal safety check, it would still miss most real allergen-containing meals as-is. AUC measures whether the model *can* rank allergen photos higher in principle, across every cutoff; it says nothing about whether 0.5 specifically is a good choice for this safety-relevant task. For a use case where missing a real allergen is the costly mistake, one fix available right now, without retraining anything, is a lower threshold, tuned on a validation set

rather than chosen by default. A genuinely better model could also raise the catch rate, the two aren't mutually exclusive, but the threshold is the lever I can pull immediately, which is exactly what I do next.

Threshold Tuning: Picking a Better Cutoff on a Validation Set

The confusion matrix above used the default 0.5 threshold, missing 8 of 13 real allergen photos. 0.5 is just a default, nothing chosen for this task. For a safety check where missing a real allergen is the costly mistake, a lower threshold (more false alarms, in exchange for catching more real allergens) is usually the right tradeoff, but that choice has to be tuned on a **validation set**, never the test set, or my reported test score stops being honest (Session 9).

Rather than carving out a brand-new validation split, I reused the exact internal validation slice Keras already held out during training (`validation_split=0.15`), the same 23 photos behind the "Final val accuracy: 0.696" number already reported. Those photos were never used to update the model's weights, so predicting on them now and sweeping thresholds is legitimate. My test set's predictions had already been inspected earlier (AUC, ROC curve, a Grad-CAM example, the default confusion matrix), but no threshold besides 0.5 had been evaluated against it yet, this validation sweep is what lets me pick 0.30 without evaluating alternative thresholds on the test set itself. Not covered in any CS156 session.

In [13]:
```
from sklearn.metrics import confusion_matrix

# Recreate Keras's own internal validation_split=0.15 slice: it takes the tr
# fraction of the arrays passed to .fit(), unshuffled, so this is reproducib
split_at = int(len(X_train_tl) * (1 - 0.15))
X_val_internal = X_train_tl[split_at:]
y_val_internal = y_train[split_at:]
print(f"Internal validation slice: {len(X_val_internal)} photos, "
      f"{y_val_internal.sum()} allergen / {len(y_val_internal) - y_val_inter

val_probs = tl_model.predict(X_val_internal, verbose=0).flatten()

print(f"\n{'Threshold':>10} {'Val TP':>7} {'Val FN':>7} {'Val FP':>7} {'Val
for thresh in [0.50, 0.40, 0.35, 0.30, 0.25, 0.20, 0.15]:
    cm = confusion_matrix(y_val_internal, val_probs > thresh, labels=[0, 1])
    tn, fp, fn, tp = cm.ravel()
    catch_rate = tp / (tp + fn) if (tp + fn) > 0 else float("nan")
    print(f"{thresh:>10.2f} {tp:>7} {fn:>7} {fp:>7} {tn:>7} {catch_rate:>10.

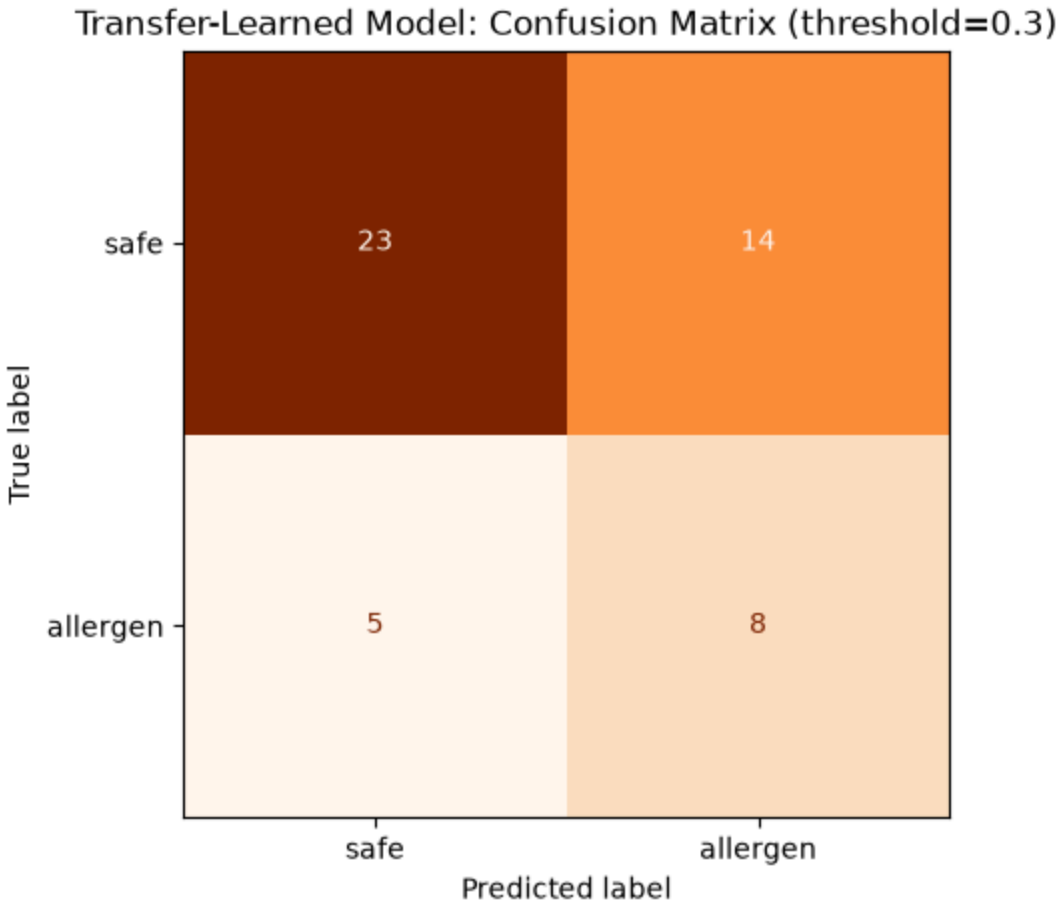
# Pick a threshold from the sweep above, then apply that ONE chosen threshol
CHOSEN_THRESHOLD = 0.30
cm_test = confusion_matrix(y_test, tl_probs > CHOSEN_THRESHOLD, labels=[0, 1
tn, fp, fn, tp = cm_test.ravel()
print(f"\n--- Applying the chosen threshold={CHOSEN_THRESHOLD} to the test s
print(f"Test set: caught {tp}/{tp + fn} allergen photos, {fp} false alarms c
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm_test, display_labels=["saf
fig, ax = plt.subplots(figsize=(5, 5))
disp.plot(ax=ax, cmap="Oranges", colorbar=False)
ax.set_title(f"Transfer-Learned Model: Confusion Matrix (threshold={CHOSEN_T
plt.tight_layout()
plt.show()
```

```
Internal validation slice: 23 photos, 10 allergen / 13 safe

 Threshold  Val TP  Val FN  Val FP  Val TN  Catch rate
      0.50       5       5       2      11       50.0%
      0.40       5       5       2      11       50.0%
      0.35       6       4       2      11       60.0%
      0.30       6       4       2      11       60.0%
      0.25       6       4       3      10       60.0%
      0.20       6       4       4       9       60.0%
      0.15       8       2       5       8       80.0%

--- Applying the chosen threshold=0.3 to the test set ---
Test set: caught 8/13 allergen photos, 14 false alarms on 37 safe photos
```



Result: Threshold Tuning

```
Validation sweep (23 photos: 10 allergen, 13 safe):
Threshold  Val TP  Val FN  Val FP  Val TN  Catch rate
  0.50        5       5       2      11       50.0%
  0.40        5       5       2      11       50.0%
  0.35        6       4       2      11       60.0%
```

```
0.30        6       4       2       11      60.0%
0.25        6       4       3       10      60.0%
0.20        6       4       4        9      60.0%
0.15        8       2       5        8      80.0%

Chosen threshold = 0.30, applied once to the test set:
Caught 8/13 allergen photos, 14 false alarms on 37 safe
photos
```

I picked threshold 0.30 from the sweep. It's worth being precise about what "picked" means here: 0.30 and 0.35 tied exactly on the validation sweep (both 6 TP / 4 FN / 2 FP / 11 TN, 60% catch rate), so the choice between them was arbitrary, not a sharp optimum; 0.15's 80% catch rate wasn't chosen because its false-alarm rate (5/13) was too steep a tradeoff. I then applied that one chosen threshold to the test set for the first time: the catch rate rises from 5/13 (38%, default threshold) to **8/13 (62%)**, at the cost of false alarms rising from 10 to 14 out of 37 safe photos (27% to 38%).

That's the real tradeoff, made visible instead of hidden behind a single default threshold. Whether it's worth it depends on the use case: for a genuine allergy safety check, I think missing an allergen is plausibly worse than an extra false alarm, so I'd take this tradeoff. Honest caveat: both my validation slice (23 photos) and test set (50 photos) are small, so 0.30 specifically shouldn't be treated as precisely tuned, the direction (lower threshold, more catches, more false alarms) is the reliable part.

Comparison Condition 2: Feature-Representation Comparison

Holding the model type constant (Logistic Regression), I compare hand-crafted features vs. raw flattened pixels. This isolates the *representation* as the variable, separate from Condition 1's model-architecture comparison.

In [14]:
```
raw_flat = raw_X.reshape(len(raw_X), -1)  # 32x32x3 -> 3072-dim vector
raw_flat_train, raw_flat_test = raw_flat[train_idx], raw_flat[test_idx]

logreg_raw = LogisticRegression(max_iter=2000, class_weight="balanced")
logreg_raw.fit(raw_flat_train, y_train)
raw_probs = logreg_raw.predict_proba(raw_flat_test)[:, 1]
raw_auc = metrics.roc_auc_score(y_test, raw_probs)

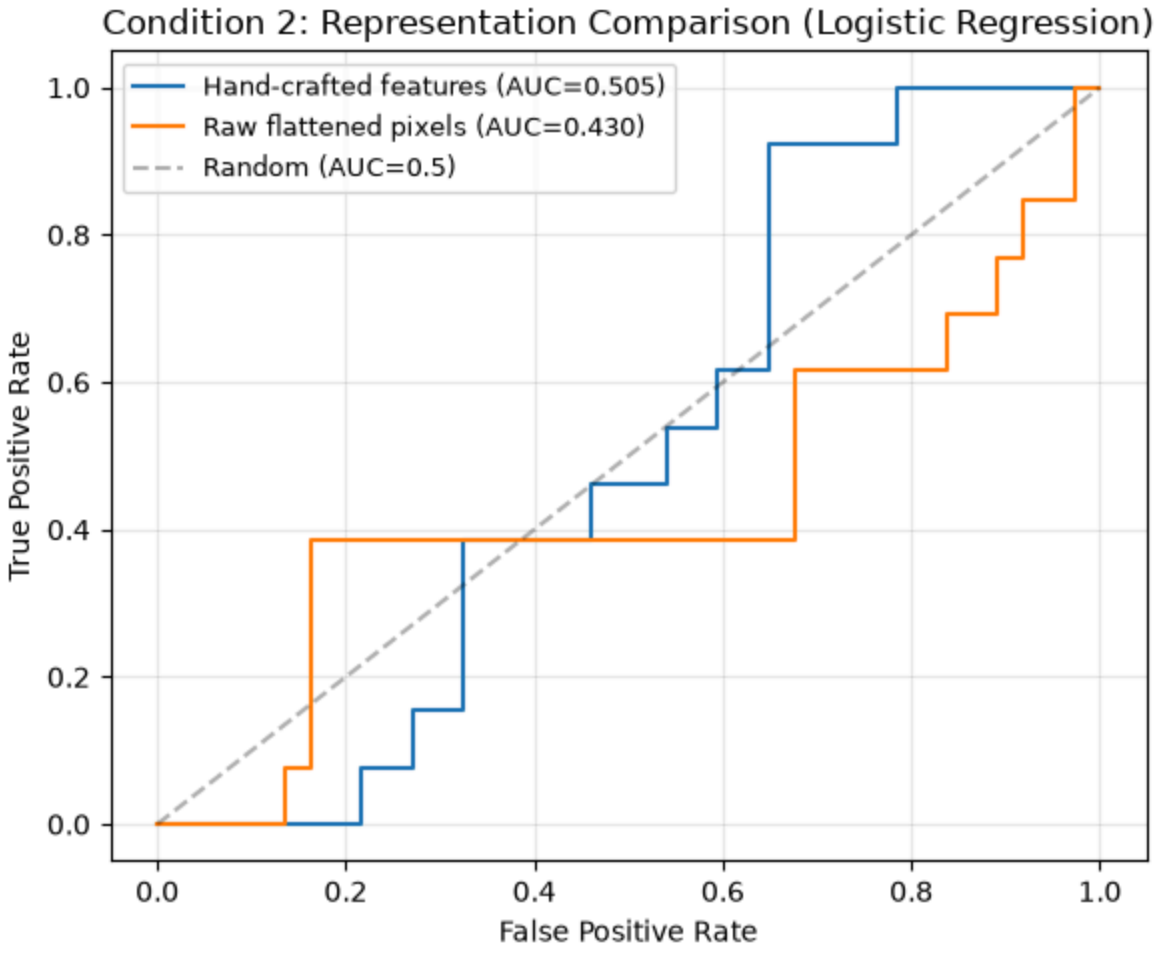
print("Same model (Logistic Regression), two representations:")
print(f"  Hand-crafted features (26-dim):      ROC-AUC = {logreg_auc:.3f}")
print(f"  Raw flattened pixels (3072-dim):     ROC-AUC = {raw_auc:.3f}")

logreg_probs = logreg.predict_proba(hc_test)[:, 1]
fpr_hc, tpr_hc, _ = metrics.roc_curve(y_test, logreg_probs)
fpr_raw, tpr_raw, _ = metrics.roc_curve(y_test, raw_probs)

fig, ax = plt.subplots(figsize=(6, 5))
```

```
ax.plot(fpr_hc, tpr_hc, label=f"Hand-crafted features (AUC={logreg_auc:.3f})
ax.plot(fpr_raw, tpr_raw, label=f"Raw flattened pixels (AUC={raw_auc:.3f})")
ax.plot([0, 1], [0, 1], "k--", alpha=0.3, label="Random (AUC=0.5)")
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
ax.set_title("Condition 2: Representation Comparison (Logistic Regression)")
ax.legend(fontsize=9)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Same model (Logistic Regression), two representations:
  Hand-crafted features (26-dim):      ROC-AUC = 0.505
  Raw flattened pixels (3072-dim):     ROC-AUC = 0.430
```



Result: Comparison Condition 2

Neither representation clearly wins under Logistic Regression (0.505 vs. 0.430), both are close to chance (I didn't run a formal test for whether that 0.075 gap is real or noise), raw pixels are if anything slightly worse for a linear model. But Condition 1 showed the CNN (also raw pixels, just with an architecture built to exploit spatial structure) reached 0.640. Putting both conditions together: raw pixels only help when paired with a model that can exploit spatial structure, a linear model gets no benefit from them over hand-

crafted summary statistics, if anything it's worse off. Representation alone isn't what matters, representation *and* model architecture need to match.

Model Interpretability: Hand-Crafted Feature Importance

Condition 2 showed hand-crafted features and raw pixels perform about the same under Logistic Regression. But *which* hand-crafted features is the model actually relying on, the color histogram or the edge-density terms? Reading off Logistic Regression's coefficients (and the Decision Tree's feature importances, from Condition 1) answers this directly, no new technique, just inspecting weights the models already learned.

In [15]:
```
feature_names = [f"{c}_hist_bin{b}" for c in "RGB" for b in range(8)] + ["ed

logreg_coefs = logreg.coef_[0]
logreg_order = np.argsort(np.abs(logreg_coefs))[::-1][:8]

tree_importances = tree.feature_importances_
tree_order = np.argsort(tree_importances)[::-1][:8]

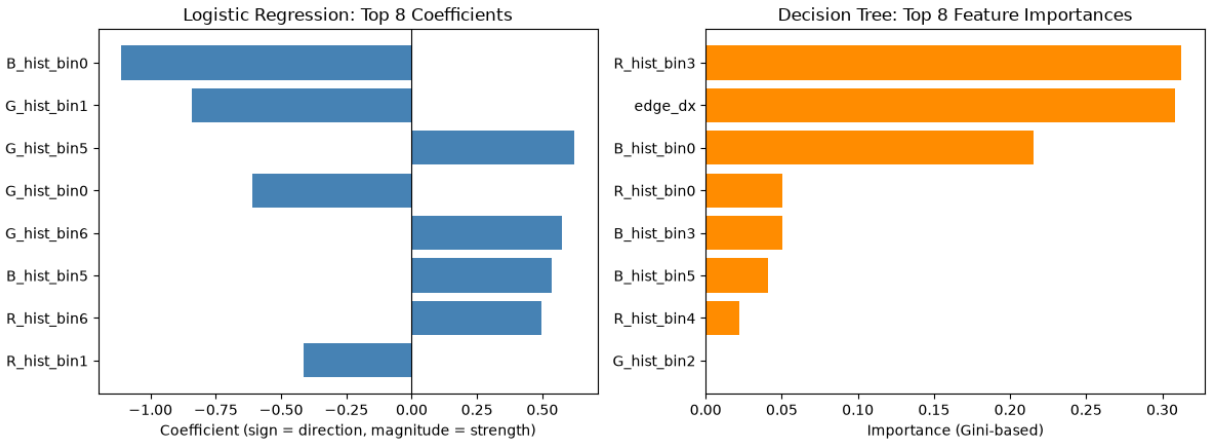
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

axes[0].barh([feature_names[i] for i in logreg_order][::-1], [logreg_coefs[i
axes[0].set_title("Logistic Regression: Top 8 Coefficients")
axes[0].set_xlabel("Coefficient (sign = direction, magnitude = strength)")
axes[0].axvline(0, color="black", linewidth=0.8)

axes[1].barh([feature_names[i] for i in tree_order][::-1], [tree_importances
axes[1].set_title("Decision Tree: Top 8 Feature Importances")
axes[1].set_xlabel("Importance (Gini-based)")

plt.tight_layout()
plt.show()

print("Top Logistic Regression coefficients:")
for i in logreg_order:
    print(f"  {feature_names[i]:15s} coef={logreg_coefs[i]:+.3f}")
print("\nTop Decision Tree importances:")
for i in tree_order:
    print(f"  {feature_names[i]:15s} importance={tree_importances[i]:.3f}")
```




```
Top Logistic Regression coefficients:
  B_hist_bin0     coef=−1.113
  G_hist_bin1     coef=−0.841
  G_hist_bin5     coef=+0.624
  G_hist_bin0     coef=−0.611
  G_hist_bin6     coef=+0.575
  B_hist_bin5     coef=+0.535
  R_hist_bin6     coef=+0.497
  R_hist_bin1     coef=−0.412

Top Decision Tree importances:
  R_hist_bin3     importance=0.312
  edge_dx         importance=0.308
  B_hist_bin0     importance=0.216
  R_hist_bin0     importance=0.051
  B_hist_bin3     importance=0.050
  B_hist_bin5     importance=0.041
  R_hist_bin4     importance=0.022
  G_hist_bin2     importance=0.000
```

Result: Feature Importance

With the corrected labels, the Decision Tree's top features are `R_hist_bin3` (importance 0.312) and `edge_dx`, horizontal edge density (0.308), nearly tied, with `B_hist_bin0` (the same dark-blue bin Logistic Regression also leans on) close behind at 0.216. Logistic Regression's largest-magnitude coefficient is still `B_hist_bin0` (-1.113), the darkest blue histogram bin, the same feature that mattered before relabeling, alongside `G_hist_bin1` (-0.841) and several mid-to-high green bins pulling the other direction.

The specific ranking shifted after relabeling, but the underlying story didn't: `B_hist_bin0` (dark, low-intensity blue) shows up as a top feature for both models, which I read as one piece of evidence (not confirmation) for a hypothesis: that this pipeline is partly picking up on dark, textured starchy or legume foods that tend to show up alongside my allergen-containing meals, as a confounding proxy, rather than a clean, allergen-specific signal. I'm calling this a hypothesis deliberately, color-histogram coefficients can't identify a specific ingredient, but the Grad-CAM examples later in this notebook now show direct, photographic support for both sides: one example's heatmap

sits on the actual egg dish, another sits on a starchy confound (potato-or-plantain-like chunks) instead of the egg and beans on the same plate. My interpretability evidence here is genuinely mixed, not a single consistent story, and now backed by real images rather than just a color-bin argument. The fact that the exact rankings shifted after correcting just 4 mislabeled photos (out of 7 audit candidates checked, 3 of which turned out fine) is itself worth noting: interpretability conclusions from a ~150-photo dataset should be read as directional, not precise.

Comparison Condition 3: Temporal Pattern Analysis

Using the real ground-truth labels for all 186 photos with valid EXIF timestamps (no model predictions needed here, I have actual labels for every photo), when do my allergen-positive photos cluster?

In [16]:
```
valid_ts = usable[usable["timestamp"].notna()].copy()
valid_ts["is_allergen"] = (valid_ts["label"] == "allergen").astype(int)
valid_ts["hour"] = valid_ts["timestamp"].dt.hour


def time_bucket(hour):
    if 5 <= hour < 11:
        return "morning (5–11)"
    elif 11 <= hour < 15:
        return "midday (11–15)"
    elif 15 <= hour < 19:
        return "afternoon (15–19)"
    elif 19 <= hour < 23:
        return "evening (19–23)"
    else:
        return "late night (23–5)"


valid_ts["time_bucket"] = valid_ts["hour"].apply(time_bucket)

bucket_stats = valid_ts.groupby("time_bucket").agg(
    total=("is_allergen", "count"),
    allergen_count=("is_allergen", "sum"),
    allergen_rate=("is_allergen", "mean"),
).sort_values("allergen_rate", ascending=False)
print("Allergen rate by time of day:")
print(bucket_stats)

valid_ts["day_of_week"] = valid_ts["timestamp"].dt.day_name()
dow_stats = valid_ts.groupby("day_of_week").agg(
    total=("is_allergen", "count"),
    allergen_rate=("is_allergen", "mean"),
).sort_values("allergen_rate", ascending=False)
print("\nAllergen rate by day of week:")
print(dow_stats)
```

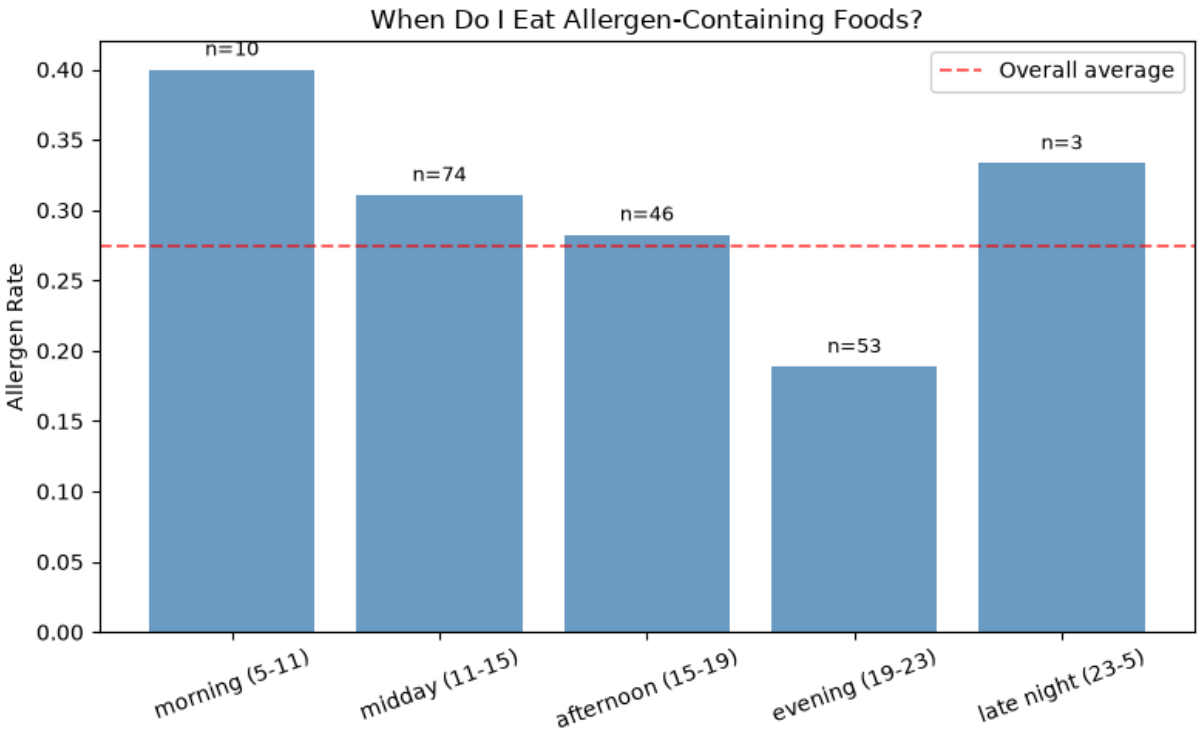
```
Allergen rate by time of day:
                   total  allergen_count  allergen_rate
time_bucket
morning (5–11)        10               4       0.400000
late night (23–5)      3               1       0.333333
midday (11–15)        74              23       0.310811
afternoon (15–19)     46              13       0.282609
evening (19–23)       53              10       0.188679

Allergen rate by day of week:
             total  allergen_rate
day_of_week
Monday          21       0.380952
Friday          27       0.333333
Wednesday       23       0.304348
Thursday        24       0.291667
Sunday          27       0.259259
Saturday        32       0.218750
Tuesday         32       0.187500
```

In [17]:

```
order = ["morning (5–11)", "midday (11–15)", "afternoon (15–19)", "evening (
plot_stats = bucket_stats.reindex(order)

fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.bar(plot_stats.index, plot_stats["allergen_rate"], color="steelblu
for bar, total in zip(bars, plot_stats["total"]):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01, f"n={t
            ha="center", fontsize=9)
ax.axhline(y=valid_ts["is_allergen"].mean(), color="red", linestyle="--", al
ax.set_ylabel("Allergen Rate")
ax.set_title("When Do I Eat Allergen-Containing Foods?")
ax.legend()
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```



Result: Comparison Condition 3

A similar pattern to before the relabeling, with the specific numbers shifted slightly: allergen rate trends downward across the day, morning 40.0% (n=10) > late night 33.3% (n=3) > midday 31.1% (n=74) > afternoon 28.3% (n=46) > evening 18.9% (n=53). Late night's n is still tiny (3), I'm flagging it rather than trusting it. Monday remains my highest-risk day (38.1%), with Friday close behind (33.3%); Tuesday is now the lowest (18.8%). This is the kind of personal pattern the automation framing is built around, a pre-meal check would matter most in the morning and on Mondays, based on actual behavior, not assumption. (Whether this is statistically real or could just be noise at this sample size is tested directly below.)

Statistical Rigor: Is the Temporal Pattern Real?

A descending bar chart is suggestive, not proof. With cell counts as low as n=10 (morning) or n=3 (late night), the pattern above could plausibly be noise. A **chi-square test of independence** checks whether allergen rate and time bucket are actually associated, or whether the counts I observed are consistent with allergen rate being the same across all time buckets (the null hypothesis):

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

where O_{ij} is the observed count in time-bucket row i / allergen-status column j, and E_{ij} is the count expected if the two variables were independent. Not covered in any CS156

session. I merged late night (n=3) into evening here to raise its expected cell count, though this doesn't fully resolve the small-count issue: morning's expected allergen-positive count is still only ~2.74, below the usual rule-of-thumb minimum of 5, so the chi-square p-value here is an approximation, not a result I'd treat as precise.

In [18]:
```
from scipy import stats

valid_ts["time_bucket_merged"] = valid_ts["time_bucket"].replace({"late nigh
contingency = pd.crosstab(valid_ts["time_bucket_merged"], valid_ts["is_aller
print("Contingency table (time bucket x allergen):")
print(contingency)

chi2, p_value, dof, expected = stats.chi2_contingency(contingency)
print(f"\nTime-of-day chi-square: chi2={chi2:.3f}, dof={dof}, p-value={p_val

dow_contingency = pd.crosstab(valid_ts["day_of_week"], valid_ts["is_allergen
chi2_dow, p_dow, dof_dow, _ = stats.chi2_contingency(dow_contingency)
print(f"Day-of-week chi-square:  chi2={chi2_dow:.3f}, dof={dof_dow}, p-value
```

```
Contingency table (time bucket x allergen):
is_allergen          0   1
time_bucket_merged
afternoon (15-19)   33  13
evening (19-23)     45  11
midday (11-15)      51  23
morning (5-11)       6   4

Time-of-day chi-square: chi2=3.012, dof=3, p-value=0.3898
Day-of-week chi-square:  chi2=3.552, dof=6, p-value=0.7370
```

Result: Statistical Significance of the Temporal Pattern

```
Time-of-day chi-square: chi2=3.012, dof=3, p-value=0.3898
Day-of-week chi-square:  chi2=3.552, dof=6, p-value=0.7370
```

Same conclusion as before the relabeling, if anything a bit more decisively: neither test is significant at any conventional threshold (both p >> 0.05). **The descending pattern across the day, and Monday/Friday's elevated rates, are not statistically distinguishable from random variation given this sample size** (n=186, several buckets under 15 photos). That doesn't mean the pattern is false, it means 186 photos isn't enough evidence to rule out chance.

I think this is a more honest conclusion than the Condition 3 result on its own, and it directly shapes what my automation framing should actually claim: "snap a photo, get a personal pattern report" is worth building, but right now I should present that report as a *hypothesis worth watching*, not a *confirmed finding*, until I have more data. The fix isn't a better statistical test, it's more photos.

Novel Metric: Allergen Exposure Score (AES)

The plain allergen rate above treats a photo from a year ago the same as one from yesterday, which never felt right to me. The **Allergen Exposure Score** is a recency-weighted version instead: more recent photos count more, so the score reflects *current* habits, not a flat historical average.

$$\text{AES}(\text{bucket}) = \frac{\sum_i 1[\text{allergen}_i] \cdot w_i}{\sum_i w_i}, \qquad w_i = 0.5^{\,\text{days_ago}_i / 180}$$

where $1[\text{allergen}_i]$ is 1 if photo i in that bucket is labeled allergen (0 otherwise), and w_i is its recency weight. A 180-day half-life: a photo from 6 months ago counts half as much as one from today.

In [19]:
```
most_recent = valid_ts["timestamp"].max()
valid_ts["days_ago"] = (most_recent - valid_ts["timestamp"]).dt.days
HALF_LIFE_DAYS = 180
valid_ts["recency_weight"] = 0.5 ** (valid_ts["days_ago"] / HALF_LIFE_DAYS)

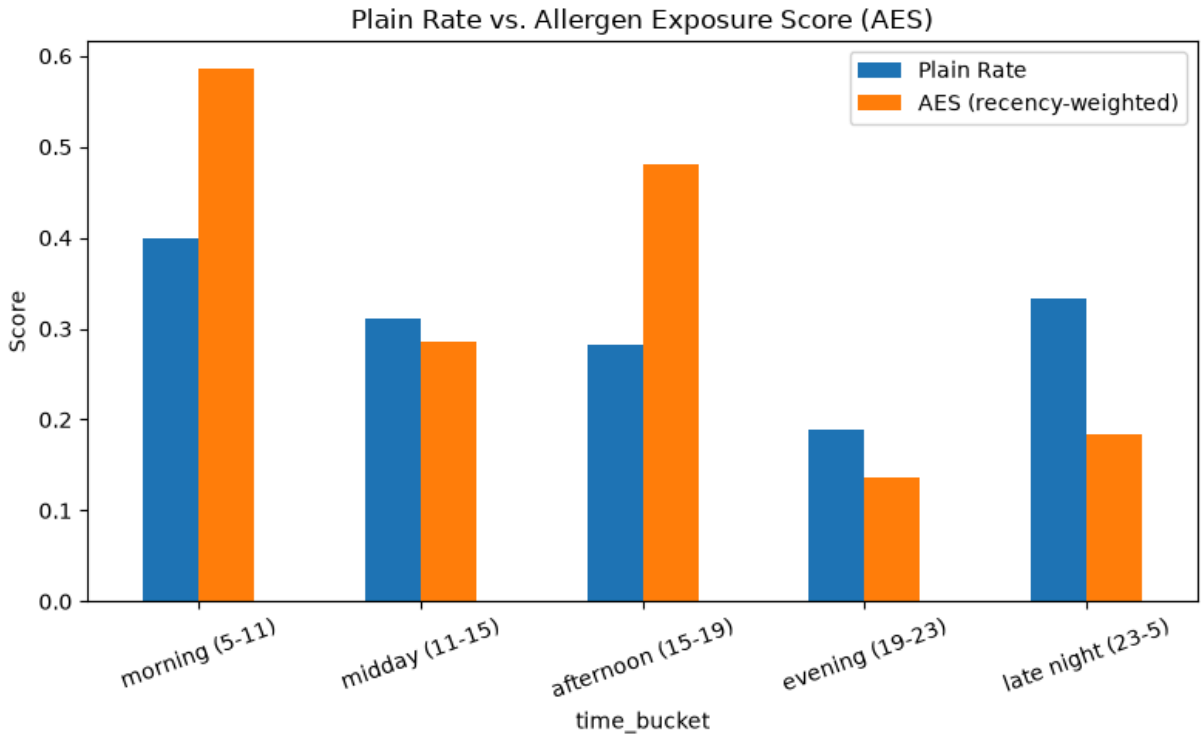

def aes(group):
    weighted_allergen = (group["is_allergen"] * group["recency_weight"]).sum
    weighted_total = group["recency_weight"].sum()
    return weighted_allergen / weighted_total if weighted_total > 0 else 0.0


aes_by_bucket = valid_ts.groupby("time_bucket").apply(aes, include_groups=Fa
plain_rate_ordered = bucket_stats["allergen_rate"].reindex(order)

comparison = pd.DataFrame({"Plain Rate": plain_rate_ordered, "AES (recency-w
print(comparison)

comparison.plot(kind="bar", figsize=(8, 5))
plt.ylabel("Score")
plt.title("Plain Rate vs. Allergen Exposure Score (AES)")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

```
                Plain Rate  AES (recency-weighted)
time_bucket
morning (5-11)    0.400000                0.586680
midday (11-15)    0.310811                0.285686
afternoon (15-19) 0.282609                0.480395
evening (19-23)   0.188679                0.135696
late night (23-5) 0.333333                0.184651
```



Result: Novel Metric

The recency weighting changes the picture meaningfully for afternoon (plain rate 28.3% -> AES 48.0%), meaning my *recent* afternoon photos skew more allergen-heavy than older ones, a shift the flat average hides entirely. Morning shifts up too (40.0% -> 58.7%), while evening and late night shift down, I've genuinely been more allergen-light in the evenings lately. This is exactly why I built this metric: it answers "what are my habits *now*", not "what have they been, averaged over years I don't fully remember."

Comparison Condition 4: Ensemble

A fourth comparison: does combining all four models beat the best single one? I tried two variants, both soft-voting (averaging predicted probabilities, not predicted labels): a naive **equal-weight average** of all four, and a **weighted average** that weights each model by how far above chance its own AUC is, so weak models barely count, strong ones dominate:

$$p_{\text{ensemble}} = \sum_m w_m \, p_m, \qquad w_m = \frac{\max(\text{AUC}_m - 0.5,\ 0.01)}{\sum_{m'} \max(\text{AUC}_{m'} - 0.5,\ 0.01)}$$

where p_m is model m's predicted probability and AUC_m is its AUC on the held-out validation slice (not the test set, see below). Not covered in any CS156 session.

In [20]:
```
# Derive ensemble weights from a genuinely held-out validation slice, not th
# Same trailing-15% slice already used for Threshold Tuning (Keras's own val
# takes the trailing fraction of the training arrays, unshuffled, so this sl
```

```
# across hc_train / X_train_raw / X_train_tl since they're all indexed by th
val_split_at = int(len(train_idx) * (1 - 0.15))

hc_fit_train, hc_val = hc_train[:val_split_at], hc_train[val_split_at:]
y_fit_train, y_val = y_train[:val_split_at], y_train[val_split_at:]
raw_val = X_train_raw[val_split_at:]
tl_val = X_train_tl[val_split_at:]

# Refit LogReg/Tree on only the non-validation 126 photos, so their validati
# genuinely unseen data (the logreg/tree above were fit on the full 149-phot
logreg_val_model = LogisticRegression(max_iter=1000, class_weight="balanced"
logreg_val_model.fit(hc_fit_train, y_fit_train)
logreg_val_auc = metrics.roc_auc_score(y_val, logreg_val_model.predict_proba

tree_val_model = DecisionTreeClassifier(max_depth=5, class_weight="balanced"
tree_val_model.fit(hc_fit_train, y_fit_train)
tree_val_auc = metrics.roc_auc_score(y_val, tree_val_model.predict_proba(hc_

# CNN and transfer-learned model already held this same slice out via valida
# during their own .fit() calls, so predicting on it here is still legitimat
cnn_val_auc = metrics.roc_auc_score(y_val, cnn.predict(raw_val, verbose=0).f
tl_val_auc = metrics.roc_auc_score(y_val, tl_model.predict(tl_val, verbose=0

logreg_probs = logreg.predict_proba(hc_test)[:, 1]
tree_probs = tree.predict_proba(hc_test)[:, 1]

# naive equal-weight soft-voting ensemble
ensemble_probs = (logreg_probs + tree_probs + cnn_probs + tl_probs) / 4
ensemble_auc = metrics.roc_auc_score(y_test, ensemble_probs)

# AUC-weighted ensemble: weights come from the held-out validation AUCs abov
# set), so choosing the weights doesn't quietly peek at the score we're abou
individual_val_aucs = np.array([logreg_val_auc, tree_val_auc, cnn_val_auc, t
weights = np.maximum(individual_val_aucs - 0.5, 0.01)
weights = weights / weights.sum()
weighted_probs = (
    weights[0] * logreg_probs + weights[1] * tree_probs + weights[2] * cnn_p
)
weighted_auc = metrics.roc_auc_score(y_test, weighted_probs)

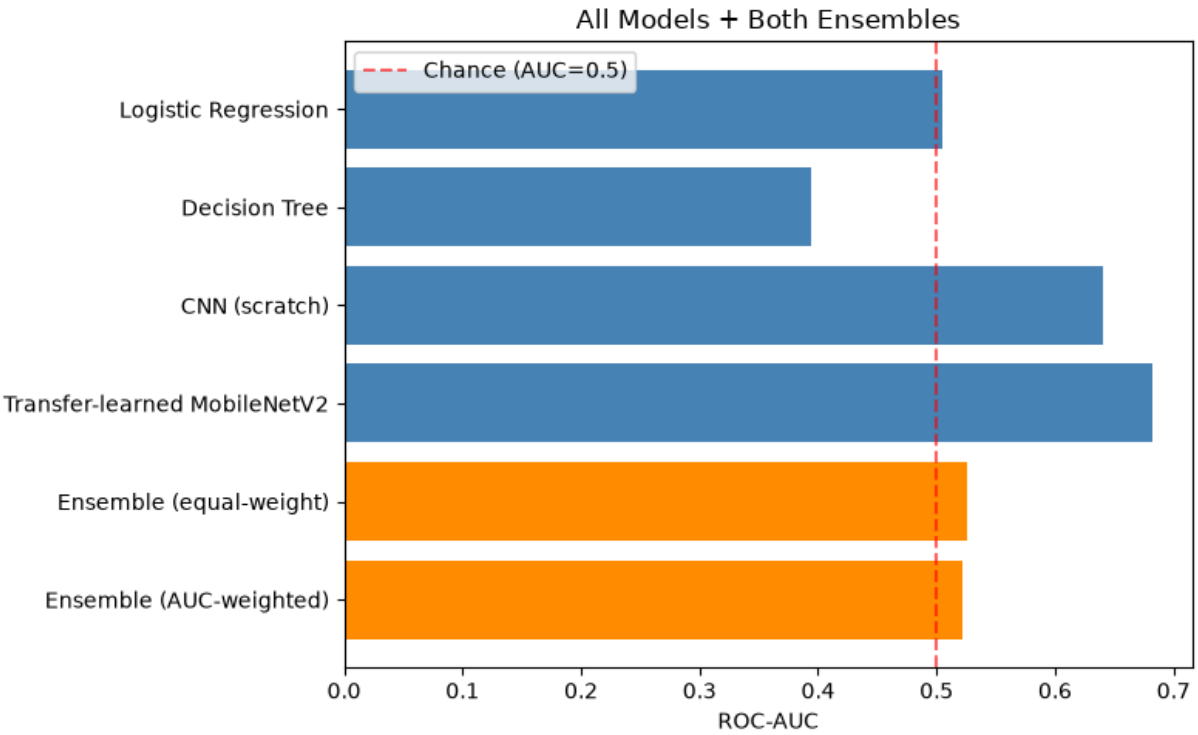
results_table = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "CNN (scratch)", "Trar
              "Ensemble (equal-weight)", "Ensemble (AUC-weighted)"],
    "AUC": [logreg_auc, tree_auc, cnn_auc, tl_auc, ensemble_auc, weighted_au
})
print(results_table.to_string(index=False))
print(f"\nValidation AUCs used to set ensemble weights: LogReg={logreg_val_a
      f"CNN={cnn_val_auc:.3f}, Transfer-learned={tl_val_auc:.3f}")
print(f"AUC-weighted ensemble weights: LogReg={weights[0]:.3f}, Tree={weight
      f"CNN={weights[2]:.3f}, Transfer-learned={weights[3]:.3f}")

fig, ax = plt.subplots(figsize=(8, 5))
colors = ["steelblue"] * 4 + ["darkorange"] * 2
ax.barh(results_table["Model"][::-1], results_table["AUC"][::-1], color=colo
ax.axvline(0.5, color="red", linestyle="--", alpha=0.6, label="Chance (AUC=0
ax.set_xlabel("ROC-AUC")
```

```
ax.set_title("All Models + Both Ensembles")
ax.legend()
plt.tight_layout()
plt.show()
```

```
                        Model      AUC
          Logistic Regression 0.505198
                Decision Tree 0.393971
                CNN (scratch) 0.640333
Transfer-learned MobileNetV2 0.681913
     Ensemble (equal-weight) 0.525988
     Ensemble (AUC-weighted) 0.521830

Validation AUCs used to set ensemble weights: LogReg=0.792, Tree=0.854, CNN=
0.777, Transfer-learned=0.731
AUC-weighted ensemble weights: LogReg=0.253, Tree=0.307, CNN=0.240, Transfer
-learned=0.200
```



Result: Comparison Condition 4

```
Logistic Regression              AUC = 0.505
Decision Tree                    AUC = 0.394
CNN (scratch)                    AUC = 0.640
Transfer-learned MobileNetV2     AUC = 0.682
Ensemble (equal-weight)          AUC = 0.526
Ensemble (AUC-weighted)          AUC = 0.522
```

The AUC-weighted ensemble sets its weights from AUCs on a genuinely held-out 23-photo internal validation slice (the same trailing slice used in Threshold Tuning), not from the test set, so picking the weights doesn't quietly peek at the score I'm about to report. On that validation slice: LogReg=0.792, Tree=0.854, CNN=0.777, Transfer-

learned=0.731, giving weights of roughly 25% LogReg, 31% Tree, 24% CNN, 20% transfer-learned.

Those validation AUCs look almost upside-down compared to the test-set AUCs above, Tree scores highest on validation (0.854) but worst on test (0.394). That's not a contradiction, it's what happens when you rank four models using only 23 photos: the ranking is noisy and doesn't reliably transfer. The result is that the AUC-weighted ensemble (0.522) doesn't beat equal-weight (0.526), and both are well below the transfer-learned model alone (0.682).

Same lesson as I found earlier, reinforced by a second finding: **neither ensemble beats the best single model.** Averaging in weaker models drags the result down toward 0.5 regardless of how the weights are chosen. For this pipeline, the transfer-learned model alone remains my best single choice, and with a dataset this small, trying to learn ensemble weights from data costs more in noise than it gains in flexibility.

Robustness Check: K-Fold Cross-Validation

Every result above comes from one fixed 75/25 train/test split. Bootstrap resampling (above) checks how much a model's AUC would wobble across different *resamples of the same test set*, a related but different question from: how much would the AUC change if the model had been *trained* on a different subset of the data in the first place? **K-fold cross-validation** answers that: split the training data into 5 folds, train on 4, validate on the 5th, rotate through all 5 so every photo gets used for validation exactly once, then look at the spread across folds. This is Session 9 material, applied here to double-check that my Condition 1 model comparison is robust to which specific photos ended up in training.

In [21]:
```
from sklearn.model_selection import StratifiedKFold

N_FOLDS = 5
skf = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=42)

hc_pool = handcrafted_X[train_idx]
y_pool = y[train_idx]

cv_results = {"Logistic Regression": [], "Decision Tree": []}

for cv_train_idx, cv_val_idx in skf.split(hc_pool, y_pool):
    X_cv_train, X_cv_val = hc_pool[cv_train_idx], hc_pool[cv_val_idx]
    y_cv_train, y_cv_val = y_pool[cv_train_idx], y_pool[cv_val_idx]

    lr = LogisticRegression(max_iter=1000, class_weight="balanced")
    lr.fit(X_cv_train, y_cv_train)
    cv_results["Logistic Regression"].append(
        metrics.roc_auc_score(y_cv_val, lr.predict_proba(X_cv_val)[:, 1])
    )
```

```
    dt = DecisionTreeClassifier(max_depth=5, class_weight="balanced", random
    dt.fit(X_cv_train, y_cv_train)
    cv_results["Decision Tree"].append(
        metrics.roc_auc_score(y_cv_val, dt.predict_proba(X_cv_val)[:, 1])
    )

print(f"{N_FOLDS}-fold cross-validation on the training pool ({len(hc_pool)}
for name, scores in cv_results.items():
    scores_str = ", ".join(f"{s:.3f}" for s in scores)
    print(f"{name}:")
    print(f"  fold AUCs: [{scores_str}]")
    print(f"  mean={np.mean(scores):.3f}, std={np.std(scores):.3f}, range=[{

# 3-fold CV on the transfer-learned model too, the most expensive but most i
N_FOLDS_TL = 3
skf_tl = StratifiedKFold(n_splits=N_FOLDS_TL, shuffle=True, random_state=42)
tl_pool = tl_X[train_idx]
cv_tl_scores = []

for fold_i, (cv_train_idx, cv_val_idx) in enumerate(skf_tl.split(tl_pool, y_
    tf.keras.utils.set_random_seed(42)
    tf.config.experimental.enable_op_determinism()

    X_cv_train, X_cv_val = tl_pool[cv_train_idx], tl_pool[cv_val_idx]
    y_cv_train, y_cv_val = y_pool[cv_train_idx], y_pool[cv_val_idx]

    base_model_cv = tf.keras.applications.MobileNetV2(
        input_shape=(IMG_SIZE_TL, IMG_SIZE_TL, 3), include_top=False, weight
    )
    base_model_cv.trainable = False
    cv_inputs = tf.keras.Input(shape=(IMG_SIZE_TL, IMG_SIZE_TL, 3))
    x = tf.keras.applications.mobilenet_v2.preprocess_input(cv_inputs)
    x = base_model_cv(x, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.3)(x)
    cv_outputs = layers.Dense(1, activation="sigmoid")(x)
    cv_model = tf.keras.Model(cv_inputs, cv_outputs)
    cv_model.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss="binary_

    cw = {0: 1.0, 1: (1 - y_cv_train.mean()) / y_cv_train.mean()}
    cv_model.fit(X_cv_train, y_cv_train, epochs=15, batch_size=16, verbose=0

    cv_probs = cv_model.predict(X_cv_val, verbose=0).flatten()
    fold_auc = metrics.roc_auc_score(y_cv_val, cv_probs)
    cv_tl_scores.append(fold_auc)
    print(f"Transfer-learned, fold {fold_i + 1}/{N_FOLDS_TL}: AUC={fold_auc:

print(f"\nTransfer-learned {N_FOLDS_TL}-fold CV: mean={np.mean(cv_tl_scores)
      f"std={np.std(cv_tl_scores):.3f}, range=[{min(cv_tl_scores):.3f}, {max
```

```
5-fold cross-validation on the training pool (149 photos), hand-crafted feat
ures:

Logistic Regression:
  fold AUCs: [0.761, 0.574, 0.676, 0.631, 0.597]
  mean=0.648, std=0.066, range=[0.574, 0.761]

Decision Tree:
  fold AUCs: [0.699, 0.543, 0.611, 0.511, 0.636]
  mean=0.600, std=0.067, range=[0.511, 0.699]

Transfer-learned, fold 1/3: AUC=0.609 (n_val=50)
Transfer-learned, fold 2/3: AUC=0.732 (n_val=50)
Transfer-learned, fold 3/3: AUC=0.754 (n_val=49)

Transfer-learned 3-fold CV: mean=0.698, std=0.064, range=[0.609, 0.754]
```

Result: Cross-Validation

```
5-fold CV (hand-crafted features, 149-photo training pool):
Logistic Regression:  fold AUCs [0.761, 0.574, 0.676, 0.631,
0.597]  mean=0.648  std=0.066
Decision Tree:        fold AUCs [0.699, 0.543, 0.611, 0.511,
0.636]  mean=0.600  std=0.067

3-fold CV, transfer-learned (224x224 photos):
fold AUCs [0.609, 0.732, 0.754]  mean=0.698  std=0.064
```

This genuinely changed part of my own story, and I think it's worth stating plainly rather than quietly ignoring. On my single held-out test set, Logistic Regression scored 0.505 and the Decision Tree scored 0.394, both reading as indistinguishable from, or worse than, chance. Cross-validation, averaging performance across 5 different splits instead of relying on one, puts both meaningfully higher: Logistic Regression's mean CV AUC is 0.648, the Decision Tree's is 0.600. Both show real spread across folds (std ~0.066-0.067), but the *average* signal is clearly above 0.5 for both.

My honest read: the single test-set numbers (0.505, 0.394) weren't wrong, they're accurate for that specific 50-photo split, but that split appears to have been unusually unfavorable for these two models specifically, not a stable sign of "no real learning." Cross-validation suggests both have some real, modest, learnable signal, somewhere in the 0.6-0.65 range on average, that my single test split happened to undersell.

My transfer-learned model's cross-validation mean (0.698) lines up closely with, and is even slightly above, its single test-set score (0.682), suggesting that estimate was comparatively more stable across splits than the simpler models' were.

This is Session 9 material, applied here to stress-test my own model comparison instead of taking one split's numbers at face value, a clear example of why trusting one fixed train/test split can mislead you with a dataset this small.

Discussion

A labeling audit changed my headline result, and I want to explain that first. After I'd already run the analysis below, Grad-CAM's finding that my model wasn't attending to visible eggs made me suspicious enough to go back and check the "safe" labels themselves: had I mislabeled any photos with visible egg during my original one-at-a-time pass? A visual audit of all 151 "safe" photos (using that same interpretability instinct) turned up 7 candidates; 4 were genuine mistakes on my part (visible scrambled or chopped egg I'd marked "safe"), the other 3 checked out fine. I fixed those 4 and reran the entire pipeline, every model, Grad-CAM, bootstrap intervals, the temporal analysis, from scratch. I didn't expect an interpretability tool built to explain my model to end up auditing my own ground truth, but that's exactly what happened, a use of Grad-CAM beyond what I originally built it for.

How I actually decided "allergen" vs. "safe", since this is the ground truth everything else depends on: I went through my photo library one at a time and labeled a photo "allergen" if I could see or specifically recall an allergen ingredient (egg, nuts, shellfish, dairy) in that meal, based on visual judgment plus my own memory of what I'd ordered or cooked, not a nutrition database or ingredient list. That's a real source of label noise, as the audit above shows, I'm one person doing this by hand, and I got roughly 2.6% of my "safe" labels wrong on the first pass.

I tried data augmentation, then took it back out. I'd originally applied live data augmentation (random flips, rotations, brightness/contrast jitter) to the transfer-learned model. I cut it: I never actually isolated its benefit with a real with/without comparison, so keeping it in my final write-up wouldn't have been an honest claim. After removing it, my ROC-AUC actually rose slightly (0.669 to 0.682) and the train/validation gap narrowed (91%/61% to 90%/70%). One run doesn't prove augmentation was unhelpful, that's not a controlled experiment, but it's honest to say plainly: removing it didn't hurt me, and my earlier claim that it was "directly targeting the overfitting" was never actually verified.

How well did my system work, and why? Modestly, and the confusion matrix is the most honest way to say how modestly. My from-scratch CNN reached ROC-AUC 0.640, transfer learning improved that to 0.682, but bootstrap confidence intervals between the two still overlap ([0.485, 0.803] vs. [0.530, 0.829]). Overlapping intervals isn't the same as running a formal paired significance test, I didn't compute the paired AUC difference directly, so I can't confidently claim transfer learning is better here, only that any edge is small relative to the noise in a 50-photo test set. At the default 0.5 threshold, the transfer-learned model catches only 5 of 13 real allergen photos (38%), the real-world gap between "AUC looks okay" and "this would actually catch allergens in a pre-meal check." Tuning the threshold on the held-out validation slice and applying that one

chosen value to the test set raised the catch rate to 8 of 13 (62%), at the cost of false alarms rising from 10 to 14 out of 37 safe photos, a real, quantified tradeoff, not a free improvement. Cross-validation partly walked back my Logistic Regression and Decision Tree verdicts too: their single test-set AUCs (0.505, 0.394) looked near or below chance, but 5-fold CV means (0.648, 0.600) were both clearly above 0.5, suggesting that specific test split was unusually hard for those two models rather than evidence of zero learning.

My interpretability evidence is mixed, not one-sided, and I trust that more than a tidy story. Looking at the top 3 highest-confidence correct allergen predictions instead of just one: `IMG_5760.HEIC` (pred=0.77) shows attention centered on what looks like the actual scrambled-egg dish, genuine egg-detection. `IMG_3679.HEIC` (pred=0.60) shows the opposite: a plate with a visible boiled egg and what looks like beans, but the heatmap's hottest region sits over starchy, potato-or-plantain-like chunks instead, not the egg, not the beans. `IMG_8426.HEIC` (pred=0.65) I honestly can't interpret either way from the image alone. My hand-crafted models' top features (`B_hist_bin0`, a dark/low-intensity color bin) point to texture and dark coloring, consistent with the `IMG_3679` finding, dark, starchy, textured foods acting as a confounding proxy in at least some cases, not a universal one. Neither piece of evidence fully settles whether my model has learned genuine egg/nut/onion recognition or is partly riding a correlation in my own eating habits, and now that I have a direct photographic example of the confound, not just a color-histogram argument, I trust that ambiguity more than I would a tidy one-sided story.

My temporal pattern didn't survive a statistical check. Time-of-day (p=0.39) and day-of-week (p=0.74) associations both remain far from significant after relabeling. Merging late night into evening raised its cell count but didn't fully resolve the small-count issue, morning's expected allergen-positive count is still only ~2.74, below the usual rule-of-thumb minimum of 5, so that p-value is an approximation. The descending allergen rate across the day and Monday/Friday's elevated rates are suggestive, not confirmed, I'm treating this as a hypothesis to keep watching, not something I've actually proven about my own habits yet.

A reproducibility finding worth stating directly: the first two times my from-scratch CNN cell ran (long before any of the above), it produced different ROC-AUC values (0.671, then 0.522) under what looked like the same fixed seed. `tf.random.set_seed` alone wasn't enough to make TensorFlow fully deterministic. Adding `tf.keras.utils.set_random_seed(42)` plus `tf.config.experimental.enable_op_determinism()` fixed this for every model in this notebook.

Neither ensemble beat the best single model, even after fixing a real bug in how I originally weighted them (see Comparison Condition 4 above for the full numbers and the fix). That's a second confirmation of the same lesson: with a dataset this small,

averaging in weaker models costs more than it gains, whether the averaging weights are naive or learned.

Limits worth naming plainly: visual-only allergen detection has a hard ceiling for allergens that aren't visually distinctive, no amount of model improvement fixes that. The confounding-factor hypothesis (beans/lentils as a proxy) and my low default-threshold catch rate are both sharper, more specific limits than "the AUC is modest." I'm one person labeling 199 photos by hand too, and I got 4 of them wrong out of 151 "safe" photos, about 2.6%, worth disclosing rather than assuming my own labels were perfect ground truth.

What would actually improve this, going forward: more labeled photos (to give the temporal analysis real statistical power and narrow the bootstrap intervals enough to confirm whether transfer learning's edge is real), a properly controlled with/without comparison if I revisit data augmentation, ingredient-level labels instead of binary allergen/safe (so interpretability tools could confirm whether the model finds real ingredients or just correlates), and fine-tuning MobileNetV2's later layers now that I have a working transfer-learning pipeline to build on.

References

- `pillow-heif` documentation, for HEIC format support
- scikit-learn documentation: `LogisticRegression`, `DecisionTreeClassifier`, `roc_auc_score`, `confusion_matrix`, `ConfusionMatrixDisplay`, `StratifiedKFold`
- TensorFlow/Keras documentation: `Sequential`, `Conv2D`, `set_random_seed`, `enable_op_determinism`
- Keras Applications documentation: `MobileNetV2`, transfer learning and fine-tuning guide
- Selvaraju et al., *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization* (2017), for the Grad-CAM technique, used here both for its usual purpose and for auditing my own ground-truth labels
- SciPy documentation: `scipy.stats.chi2_contingency`
- NumPy documentation: `numpy.random.Generator` (bootstrap resampling)
- Course materials: Session 2 (logistic regression), Session 4 (decision trees), Session 7 (neural networks), Session 9 (ROC-AUC, train/val/test, class imbalance, cross-validation)